

What is an Algorithm?

Learning Objectives

- Define algorithms and explain their role in computational thinking
- Identify the key characteristics of a well-formed algorithm
- Analyze real-world processes as sequences of algorithmic steps

Engage (5-7 minutes)

Begin with a relatable scenario: planning a morning routine. Ask students to list every step they take from waking up to arriving at school. Discuss how omitting steps (like forgetting to set an alarm) causes failures—mirroring how incomplete algorithms produce errors in computing.

Explore (10-12 minutes)

In pairs, students break down everyday tasks (making tea, tying shoes, following a recipe) into precise step-by-step instructions. Each pair writes their instructions on cards, then swaps with another pair who attempts to follow them literally. This reveals ambiguities and missing steps in poorly defined algorithms.

Explain (10-12 minutes)

An algorithm is a finite, ordered sequence of unambiguous instructions that solves a specific problem or performs a computation. Key properties include: definiteness (each step is precisely defined), input (zero or more inputs), output (at least one output), finiteness (it terminates after a finite number of steps), and effectiveness (each step is feasible). Algorithms are the foundation of all software—they precede coding and exist independently of any programming language.

Elaborate (8-10 minutes)

Examine algorithms in non-computing contexts: navigation directions, mathematical long division, and chemical procedures. Compare naive approaches versus optimized ones (e.g., finding a name in a phonebook by flipping randomly vs. binary search). Introduce Big-O notation conceptually—how we measure algorithm efficiency by counting operations relative to input size.

Evaluate (5-8 minutes)

Students write an algorithm for a simple task (e.g., calculating the average of a list of numbers) and evaluate a peer's algorithm for correctness, completeness, and ambiguity. Self-assessment: students reflect on which step was hardest to define precisely and why precision matters in computing.